Lecture 05.07: Instance Methods

📌 Executive Summary

Definition and mechanics of instance methods, self pointer binding, accessing and mutating instance state, and method chaining patterns.

🔬 Theoretical Foundations & Principles

1. Instance Method Semantics

An instance method is a function defined within a class whose first formal parameter is conventionally named `self`.

When invoked via `obj.method(arg)`:

1. CPython evaluates descriptor `Class.__dict__['method'].__get__(obj, Class)`.
2. This creates a callable `types.MethodType` object binding `self = obj`.

2. Method Chaining Pattern

Returning `self` at the termination of mutator methods enables fluent interface chaining: `builder.set_a().set_b().build()`.

💻 Production Implementation & Code

```
class Vector2D:
    def __init__(self, x: float = 0.0, y: float = 0.0):
        self.x = x
        self.y = y

    def translate(self, dx: float, dy: float):
        self.x += dx
        self.y += dy
        return self  # Return self for chaining

    def scale(self, factor: float):
        self.x *= factor
        self.y *= factor
        return self
```

```
v = Vector2D(1, 2)
v.translate(3, 4).scale(2)  # Method chaining
print(f"Chained Vector Result: ({v.x}, {v.y})")
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Method dispatch overhead is $\mathcal{O}(1)$ via CPython descriptor resolution.
- **Auxiliary Space:** Instance attribute memory allocation in `__dict__` vs `__slots__`.